Operating Systems
Chapter 3

Process Description and Control
(Processes Management)

Instructor

Dr.Eng. Mahmoud M. Ismail

Contact: dr.eng.mahmoudmmi@gmail.com
dr.mahmoudmmi@yahoo.com

Outline

- Process States
- Process Description
- Process Control
- Execution of the Operating System

Process

- **Program is passive entity stored on disk**
- ***Several definitions of* the process:**
  - **A program in execution (A process is active)**
  - **– Program becomes process when executable file loaded into memory**
  - **– Execution of program started via GUI mouse clicks, command line entry of its name.**
  - **An instance of a program running on a computer.**
  - **The entity that can be assigned to and executed on a processor.**
  - **A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources.**
- **Also called a task**
- **Can be traced**
  - **– list the sequence of instructions that execute for that process (*process trace*)**

Program vs Process

Program	Process
A Program is basically **a collection of instructions** that mainly performs a specific task when executed by the computer.	The process is basically **an instance of the program that is being executed**.
A Program has **a longer lifetime**.	A process has **a shorter lifetime**.
A Program is **stored by hard-disk** and **does not require any resources**.	A Process **requires resources** such as memory, CPU, Input-Output devices.
A Program has **static code** and **static data**.	A process has **a dynamic instance of code and data**
On the other hand, the program is the **executable code**.	Basically, a process is the **running instance of the code**.

Major Requirements of an Operating System
Requirements that the OS must meet can be expressed with reference to processes

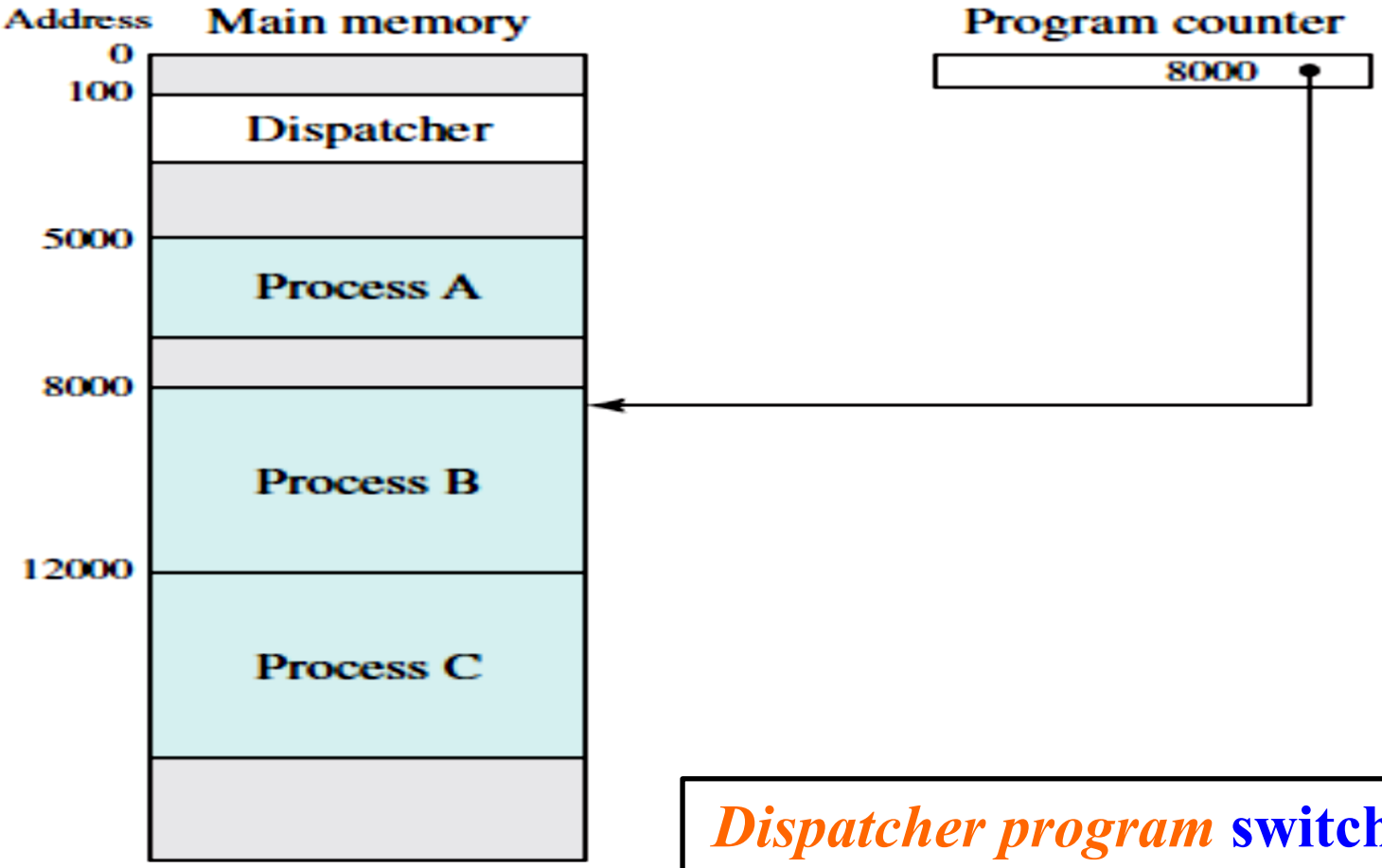
1) Interleave the execution of multiple processes (alternating between them), to maximize processor utilization while providing reasonable response time.

2) Allocate resources to processes in conformance with a specific policy (e.g., Certain applications are of higher priority) while at the same time avoiding deadlock.

3) Support **interprocess communication** and **user creation of processes**.

Process trace

List the sequence of instructions that execute for that **process**.



Dispatcher program **switches the processor from one process to another**

Figure 3.1 Snapshot of Example Execution (Figure 3.3) at Instruction Cycle 13

- We can characterize behavior of the processor by showing **how the traces of the various processes** are **interleaved.**

Process A	Process B	Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of process A (b) Trace of process B (c) Trace of process C

5000 = Starting address of program of process A
8000 = Starting address of program of process B
12000 = Starting address of program of process C

Figure 3.2 Traces of Processes of Figure 3.1

1	5000		27	12004	
2	5001		28	12005	
3	5002	A	----------Time-out		
4	5003		29	100	
5	5004		30	101	
6	5005		31	102	
----------Time-out			32	103	
7	100		33	104	
8	101		34	105	
9	102		35	5006	
10	]103		36	5007	
11	]104		37	5008	A
12	105		38	5009	
13	8000		39	5010	
14	8001	B	40	5011	
15	8002		----------Time-out		
16	8003		41	100	
----------I/O request			42	101	
17	100		43	102	
18	101		44	103	
19	102		45	104	
20	103		46	105	
21	104		47	12006	
22	105		48	12007	
23	12000		49	12008	C
24	12001	C	50	12009	
25	12002		51	12010	
26	12003		52	12011	
			----------Time-out		

We assume the OS only allows a process to continue execution for a maximum of six instruction cycles, after which it is interrupted; this prevents any single process from monopolizing processor time.

The processor stops executing process **B** and moves on, via the dispatcher, to process **C**.

100 = Starting address of ***dispatcher program***
Shaded areas indicate execution of ***dispatcher process***
first and third columns count instruction cycles
second and fourth columns show address of instruction being executed.

Figure 3.3 Combined Trace of Processes of Figure 3.1

Reasons for Process Creation

Submission of a batch job	When the OS is prepared to take on new work, it will **read the next sequence of job control commands**.
Interactive log-on	A user at a terminal **logs on to the system**.
Created by OS to provide a service such as printing	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g. **a process to control printing**).
Process creates another process	• An application process may generate another process to receive data that the application is generating, and to organize those data into a form suitable for later analysis. • The new process runs in **parallel** to the original process and is activated from time to time when new data are available.

Process Termination

- Batch job issues *Halt* instruction (generate an interrupt to alert the OS that a process has completed.)
- User logs off
- Quit an application (e.g., word processing or spreadsheet)
- Error and fault conditions

Reasons for Process Termination

- Normal completion
- Time limit exceeded
  - When the process execution does not complete within the **time quantum**
- Memory unavailable
- Bounds violation
  - Access a memory location that it is not allowed to access
- Protection error
  - example write to read-only file
- Arithmetic error (such as division by zero)
- Time overrun
  - Process waited longer than a specified maximum for an event to occur.

Reasons for Process Termination

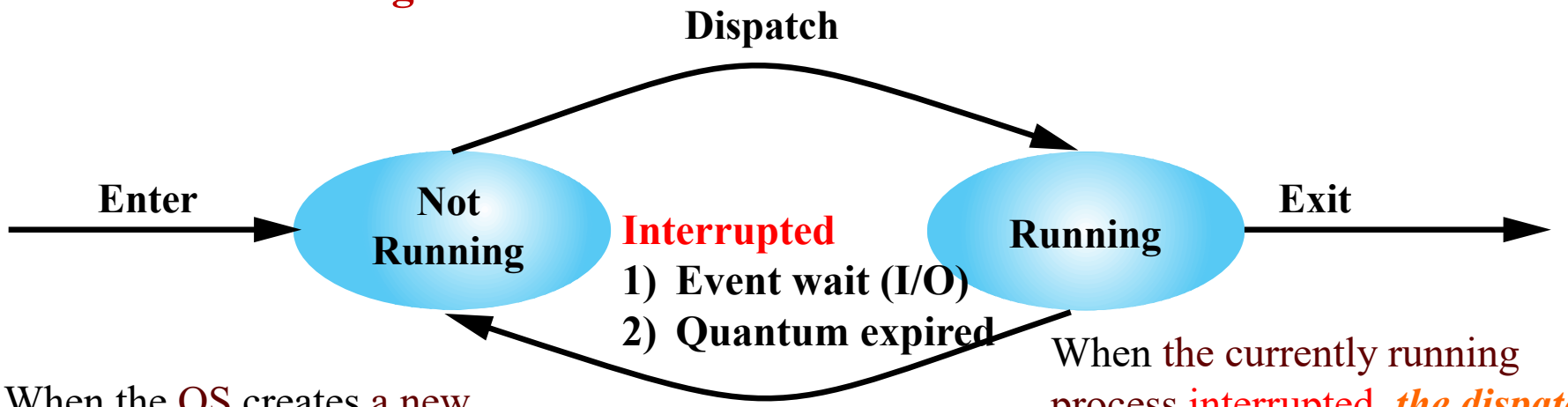
- I/O failure
  - Inability to find a file, failure to read or write after a specified maximum number of tries (a defective area is encountered on a tape).
- Invalid instruction
  - Often a result of branching into a data area and attempting to execute the data (stack overflow)
- Privileged instruction
  - The process attempts to use an instruction reserved for the OS.
- Data misuse
  - A piece of data is of the wrong type or is not initialized

Reasons for Process Termination

- Operating system intervention
  - such as when deadlock occurs
- Parent termination
  - When a parent terminates, the operating system may automatically terminate child processes .
- Parent request
  - A parent process typically has the authority to terminate any of its child processes.

Two-State Process Model

- Process may be in one of two states
  - **Running**
  - **Not-running**



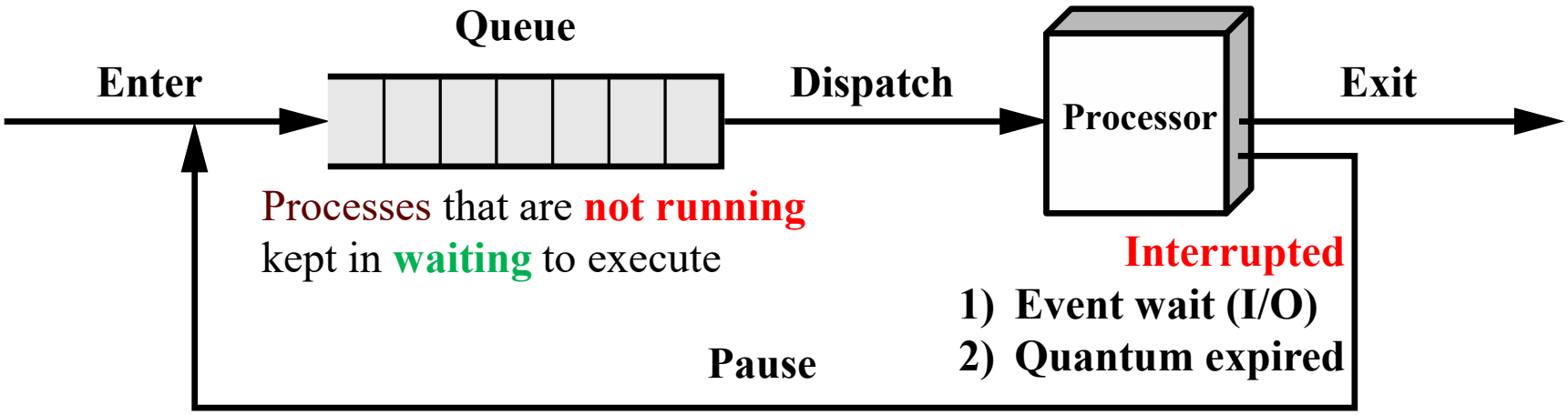
When the OS creates a new process, enters that process in the **Not Running state** and is **waiting to execute**.

When the currently running process interrupted, ***the dispatcher*** (portion of the OS) moves it from Running state to Not Running state, and one of the other processes moves to the Running state.

(a) State transition diagram

Figure 3.4 Two-State Process Model

Not-Running Process in a Queue



(b) Queueing diagram

Figure 3.4 Two-State Process Model

A process that is interrupted is transferred to **the queue** of waiting processes.

There is **a single queue** in which each entry is **a pointer** to the process control block (current state and location in memory) of a particular process

Processes in the Not Running state

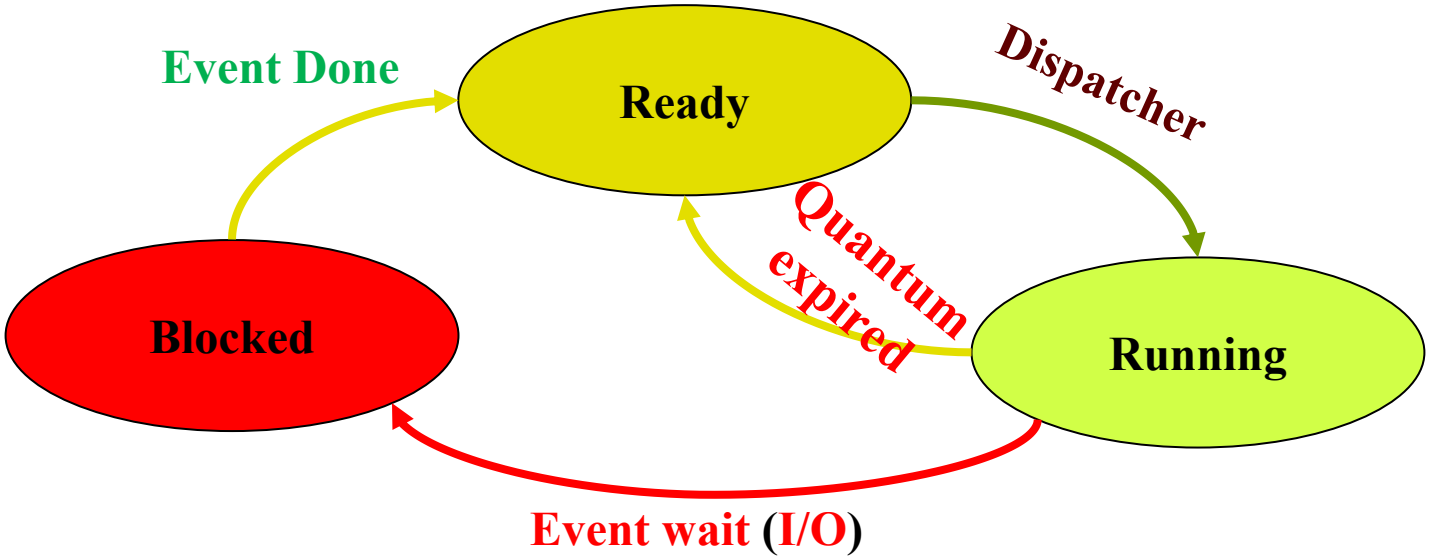
The previous implementation is inadequate: Some processes in the Not Running state are ready to execute, while others are blocked, waiting for an I/O operation.

✓ To handle this situation

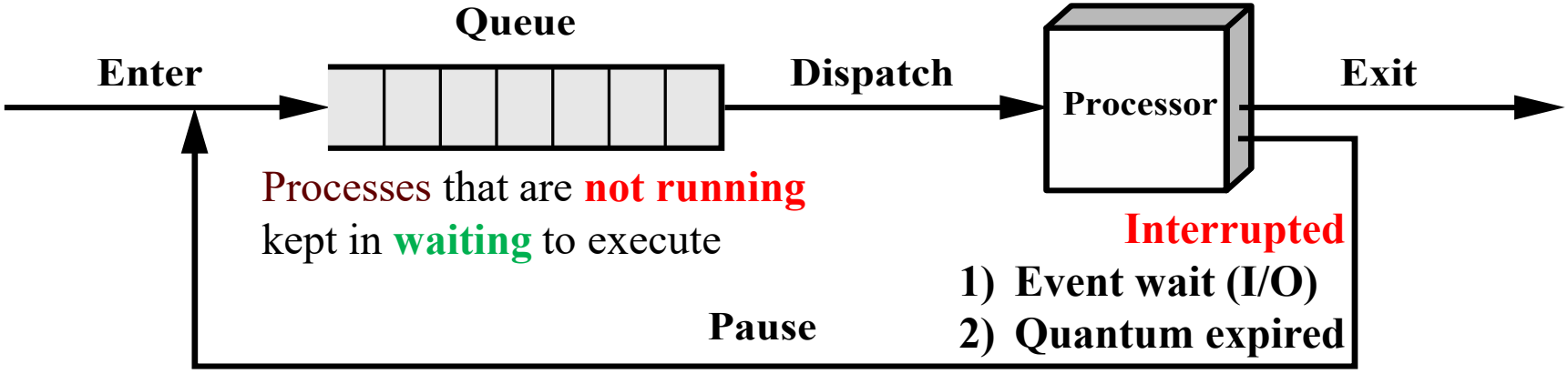
➢ Split the Not Running state into two states:

1) **Ready** to execute

2) **Blocked** (waiting for I/O to complete)



Processes in the Not Running state



A process that is interrupted is transferred to **the queue** of waiting processes.

- The queue is a first-in-first-out list and the processor operates in round-robin fashion on the available processes.
  - Each process in **the queue** is given **a certain amount of time**, in turn, to execute and then returned to **the queue**, **unless** blocked.

- **Thus**, using **a single queue**, Dispatcher cannot just select the process that has been in **the queue** the longest because it may be **blocked**.

A Five-State Model

- New
- Ready
- Running
- Blocked/Waiting
- Exit

Five-State Process Model

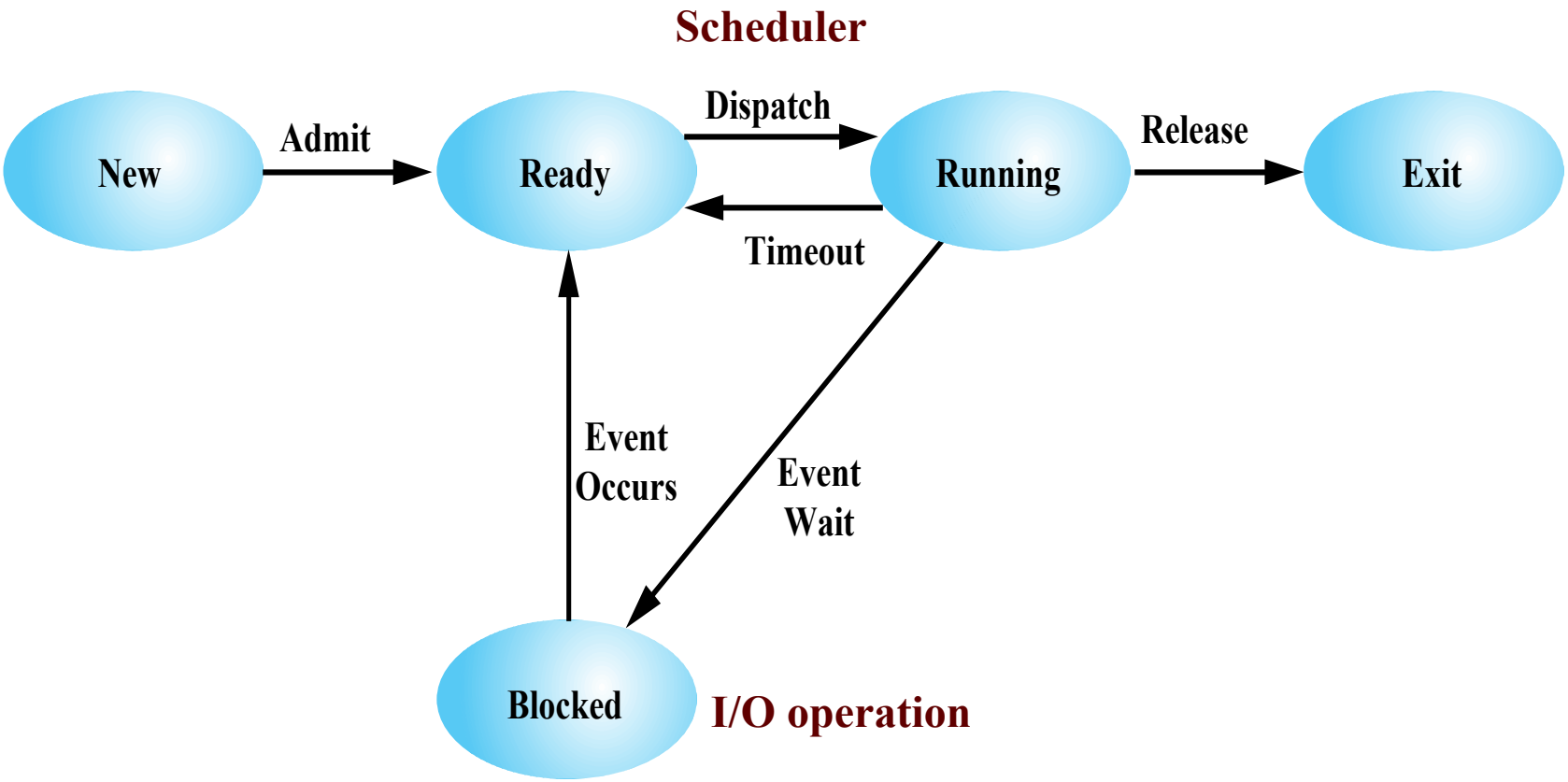


Figure 3.5 Five-State Process Model

A Five-State Model

- **New:** A process that has just been created **but** has not yet been admitted to the pool of executable processes by the OS.
  - Typically, a new process has not yet been loaded into main memory
- **Ready: The process is waiting to be assigned to a processor**
- **Running: The process is currently being executed**
- **Blocked/Waiting: The process is waiting for some event to occur (such as an I/O completion).**
- **Exit/Terminated: The process has finished execution or aborted for some reason**

The types of events that lead to each state transition for a process

1) New: A new process is created to execute a program.

2) New → Ready: The OS will move a process from the New state to the Ready state when it is prepared to take on an additional process.

   ➢ Most systems set some limit based on the number of existing processes or the amount of virtual memory committed to existing processes.

   - This limit assures there are not so many active processes as to degrade performance.

3) Ready → Running: select a process to run, the OS chooses one of the processes in the Ready state (which one?). This is the job of the scheduler or dispatcher.

4) Running → Exit: The currently running process is terminated by the OS if the process indicates that it has completed or if it aborts.

5) **Running → Ready: The most common reasons for this transition are:**

1) **The running process has reached the maximum allowable time.**

2) **The OS assigns different levels of priority to different processes.**

- **Suppose, for example, process A is running at a given priority level, and process B, at a higher priority level, is blocked. If the OS learns that the event upon which process B has been waiting has occurred, this moving B to a ready state, then it can interrupt process A and dispatch process B. We say that the OS has preempted process A.**

6) **Running → Blocked/Waiting: A process is put in the Waiting state if it requests something for which it must wait.**

- ✓ **A request to the OS is usually in the form of a system service call; that is, a call from the running program to a procedure that is part of the operating system code.**

- ➢ **The process may initiate an action, such as I/O operation, that must be completed before the process can continue.**
- ➢ **Or The process may request a service from the OS that the OS is not prepared to perform immediately. It can request a resource, such as a file or a shared section of virtual memory, that is not immediately available.**
- ➢ **Or when processes communicate with each other, a process may be blocked when it is waiting for another process to provide data, or waiting for a message from another process.**

7) **Blocked/Waiting → Ready: A process in the waiting state is moved to the Ready state when the event for which it has been waiting occurs.**

8) **Ready → Exit: this transition is not shown on the state diagram.**

- **In some systems, a parent may terminate a child process at any time. Also, if a parent terminates, all child processes associated with that parent may be terminated.**

9) **Blocked/Waiting → Exit: The comments under the preceding item apply.**

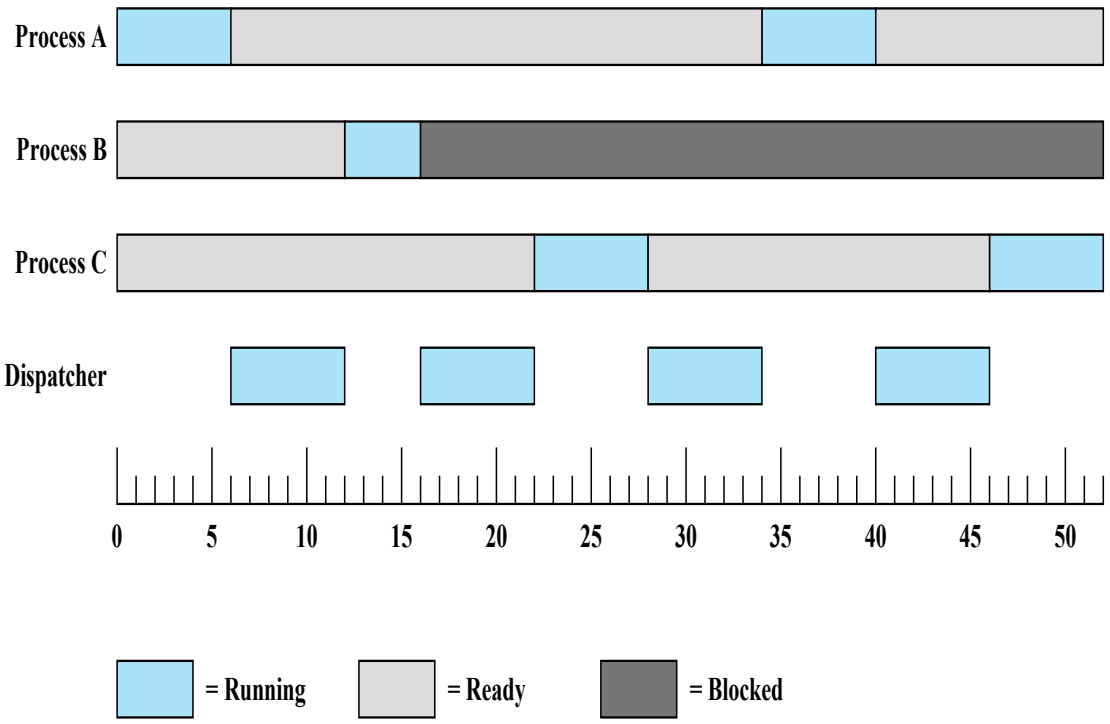
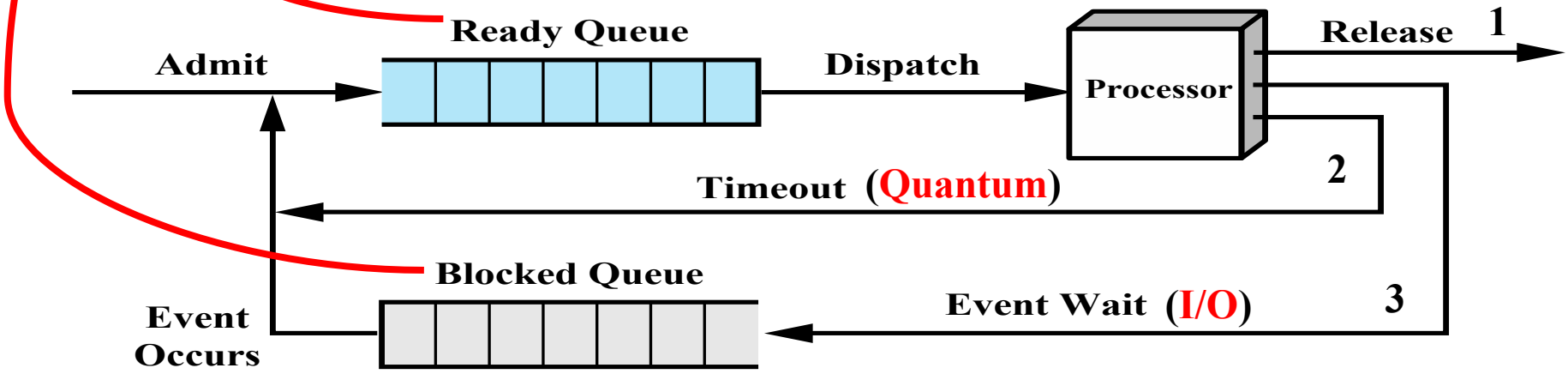


Figure 3.6 Process States for the Trace of Figure 3.3

1	5000	
2	5001	
3	5002	A
4	5003	
5	5004	
6	5005	
----------	Time-out	
7	100	
8	101	
9	102	
10	103	
11	104	
12	105	
13	8000	
14	8001	B
15	8002	
16	8003	
----------	I/O request	
17	100	
18	101	
19	102	
20	103	
21	104	
22	105	
23	12000	
24	12001	C
25	12002	
26	12003	
27	12004	
28	12005	
----------	Time-out	
29	100	
30	101	
31	102	
32	103	
33	104	
34	105	
35	5006	
36	5007	
37	5008	A
38	5009	
39	5010	
40	5011	
----------	Time-out	
41	100	
42	101	
43	102	
44	103	
45	104	
46	105	
47	12006	
48	12007	
49	12008	C
50	12009	
51	12010	
52	12011	
----------	Time-out	

Queue or Linked List ?

Using Two Queues



(a) Single blocked queue

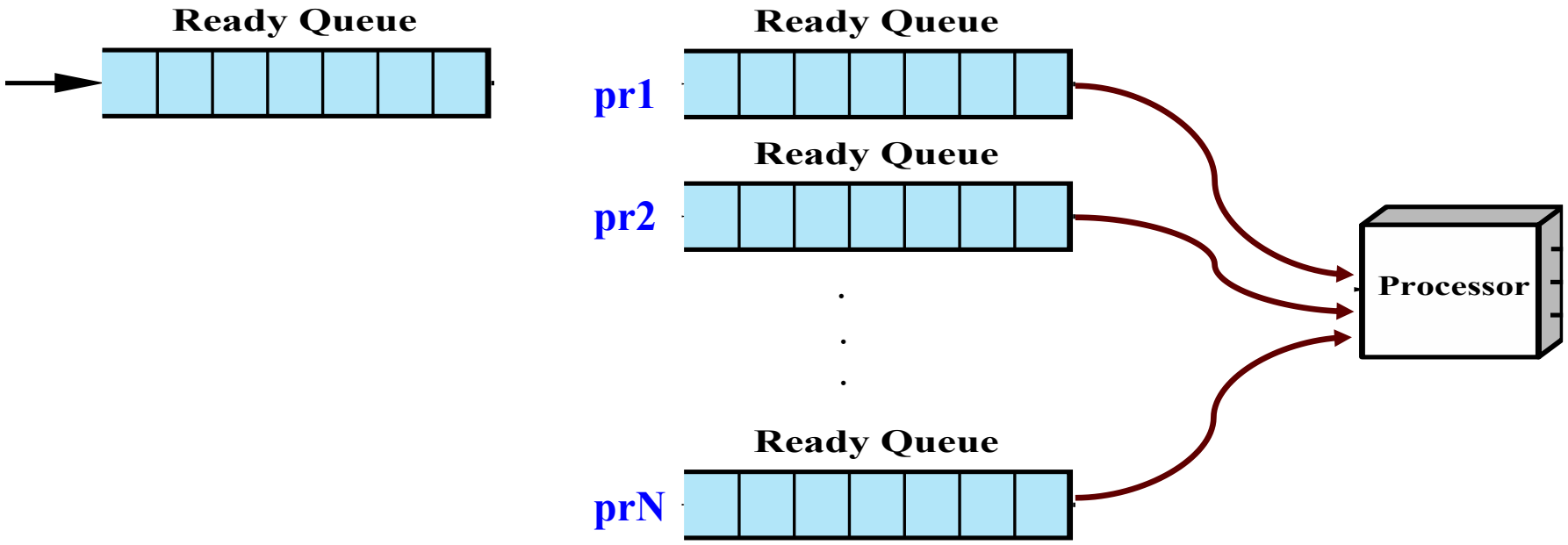
Figure 3.7 Queueing Model for Figure 3.5

- Each process admitted to the system, it is placed in the **Ready queue** (**absence of priority**)
- OS chooses process from the Ready queue to run (first-in-first-out)
- A running process is removed from execution
  1) **terminated**
  2) **or placed in the ready queue (timeout)**
  3) **or blocked queue (I/O operation)**

- If dispatching of processes dictated by **a priority** scheme, can have **a number of ready queues**, one for each **priority level**.
- **When an event** occurs, **all processes** in the **blocked queue** that are waiting on **that event** are moved to the **ready queue** → **not efficient why?**

Using Multiple Ready Queues for each priority level

Assume OS with priorities ranges from pr1 … prN such that pr1 > pr2 > … prN



Disadvantages

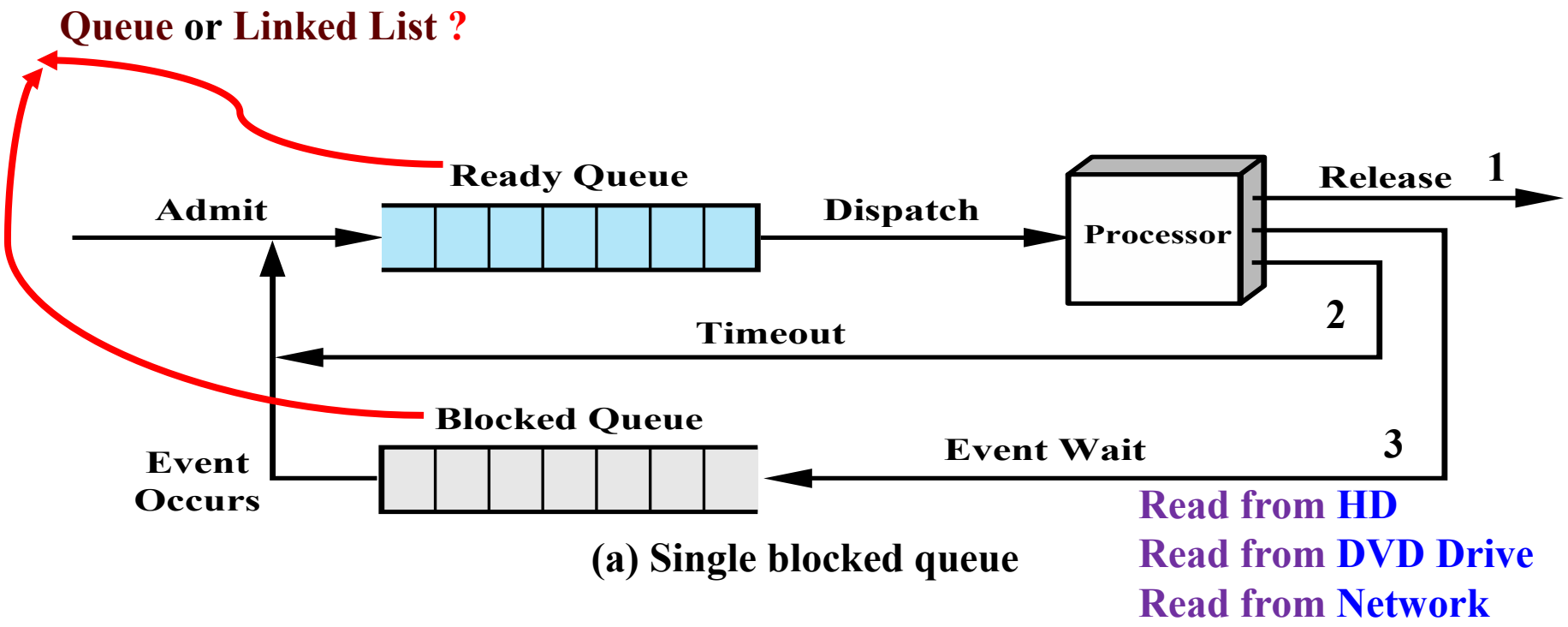
Starvation: Processes in **prN** (**Least priority**) most probably never get processor (**unfairness**)

Solution: **Aging** after certain time **gradually increase the priority**, Processes in **prN** moved from **prN** to **prN-1**. **Eventually** reached to **pr1**.

Solution can be customized such that after two minutes, Processes in **prN** moved to **prN-1** and after one minute moved to **prN-1** and so on.

Eventually, there is **guarantee** that Processes in **prN** (**Least priority**) **will get** processor

Using Multiple Blocked Queues for each event



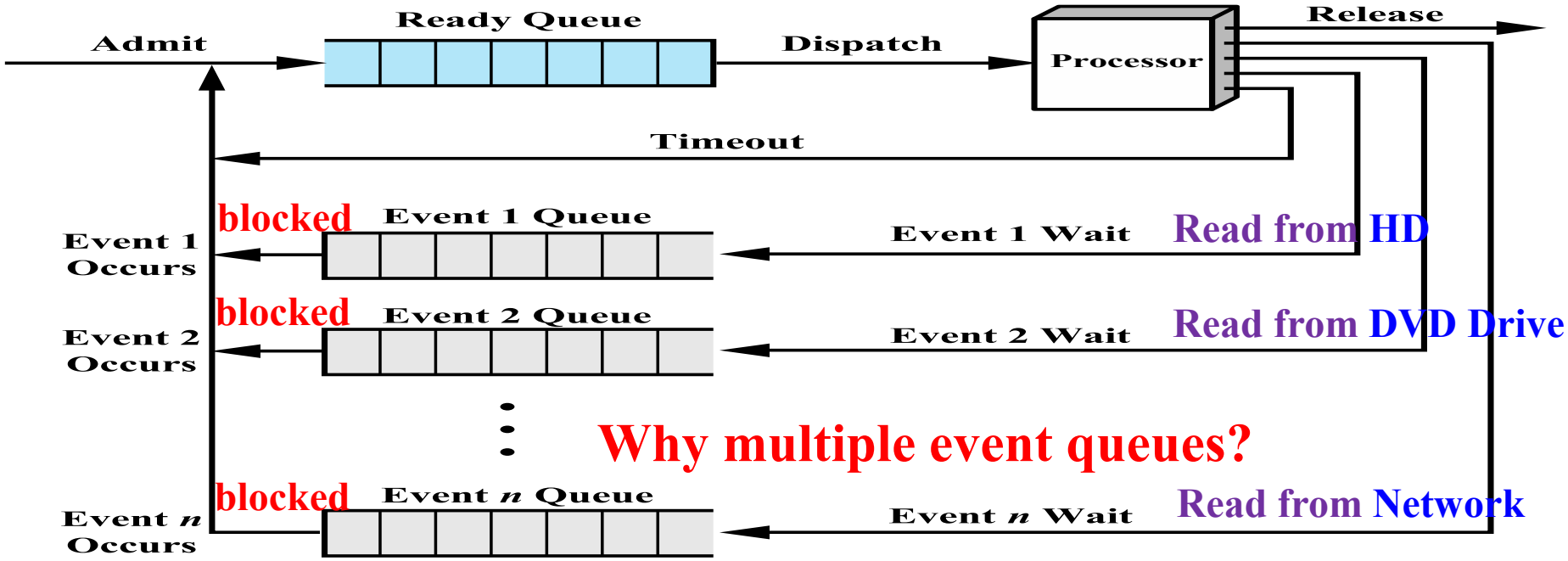
(a) Single blocked queue

➢ **When an event** occurs, **all processes** in the **blocked queue** that are waiting on **that event** are moved to the **ready queue** → **not efficient why?**

- When **an event** occurs, OS must scan the entire blocked queue, searching for those **processes** waiting **that event** (there could be hundreds or thousands of processes in that **blocked queue**)
- **Therefore**, it would be more efficient to have a number of **blocked queues**, one for **each event**.

✓ **Then**, when the **event** occurs, all of processes in the appropriate **queue** can be moved to the **Ready queue**.

Using Multiple Blocked Queues for each event



(b) Multiple blocked queues

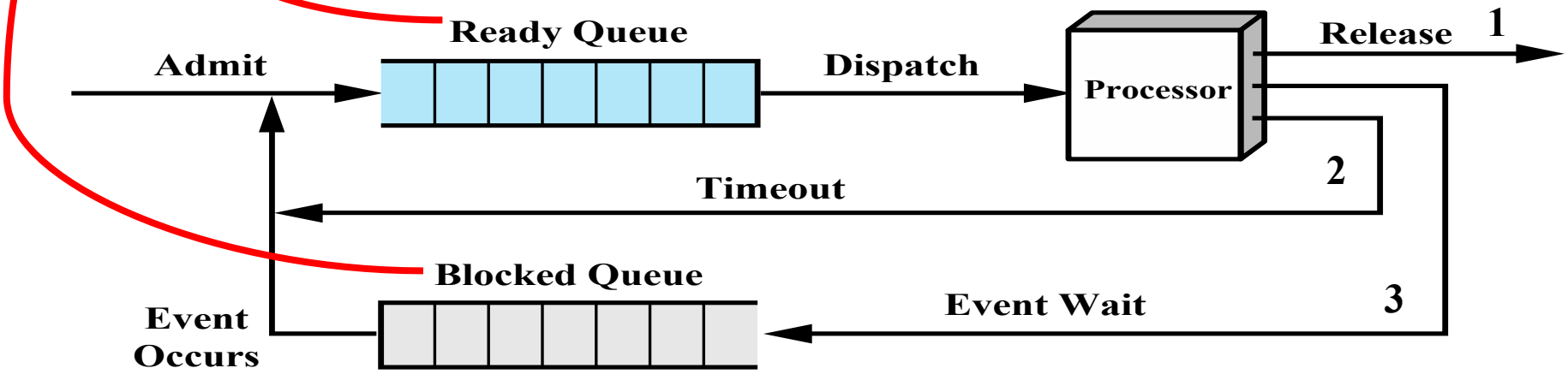
Figure 3.7 Queueing Model for Figure 3.5

- **Therefore**, it would be more efficient to have a number of **blocked queues**, one for **each event**.

✓ **Then**, when the **event** occurs, all of processes in the appropriate **queue** can be moved to the **Ready queue**.

Queue or Linked List ?

Using Two Queues



(a) Single blocked queue

Ready Queue : Queue or **Linked List** ?

Based on **Scheduling Criteria**

If based on **arrival time** → **Queue**

If based on **least execution time** → **Linked List** (sorted on **least execution time**)

Blocked Queue : are the **Blocked** processes can be maintained in **Queue** or **Linked List** ?

Read from HD, DVD Drive, Network. Is there any guarantee that occurred in FIFO?

So, Linked List is used.

Suspended Processes:The need for swapping

- Each process to be executed **must be loaded fully** into main memory
- Memory holds multiple processes and the processor can move to another process when one process is blocked.
  - **But** Processor is faster than **I/O** so, all processes could be waiting for **I/O**.
  - **Thus**, a processor could be **idle** most of the time.

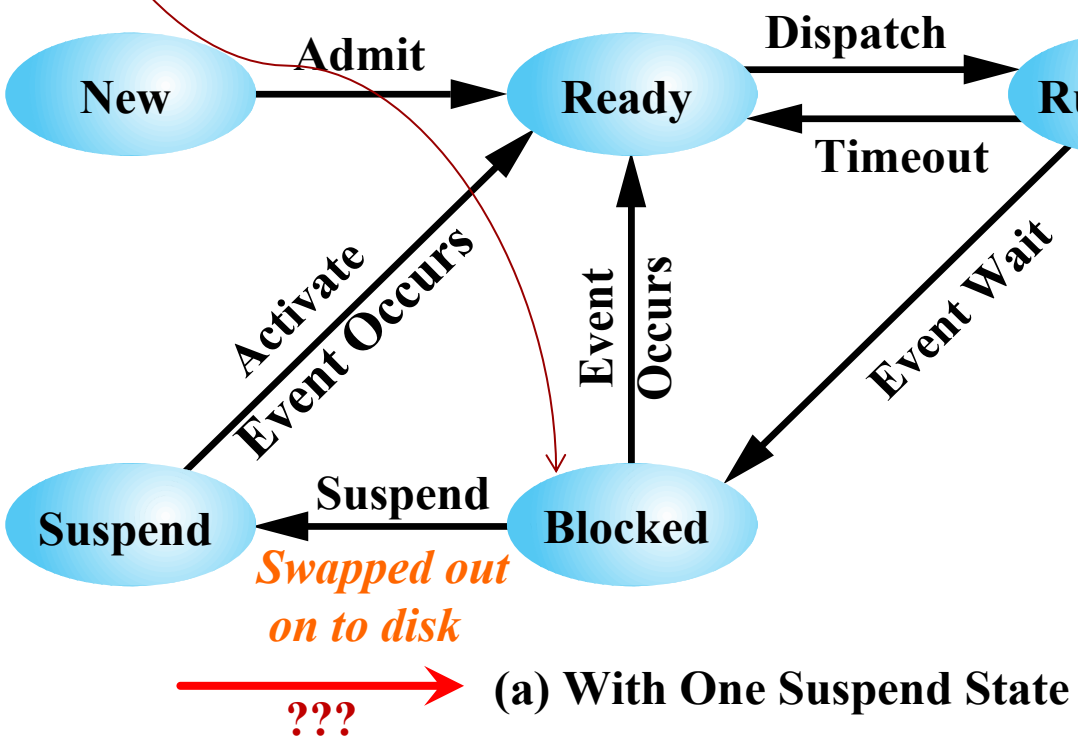
❖ Process State Transition Diagram **did not exploit secondary storage in case of no enough memory for new process.**

❖ What to do?

1) One Solution main memory could be **expanded** to accommodate more processes.
2) Another solution is ***swapping***, which involves ***Swap*** these processes (**blocked**) to **disk** to free up more memory for higher priority process (**moving all parts of process** from main memory to **disk**)

One Suspend State

- With the use of ***swapping***, one other state must be added the ***Suspend state***
- When some or all of the processes in main memory are in the **Blocked state**, the OS **suspend** one of the Blocked process (***swapped out***) by putting it in the **Suspend state** and **transferring** it to **disk**. (Why from **Blocked state?**)
- The space that is freed in main memory can then be used to bring in another process.



How to manage processes in ***Suspend state?(data structure)***
Is queue?
No control on timing i.e. which event occurred first.
***So, general list* is used**

The objective: is to free up more memory

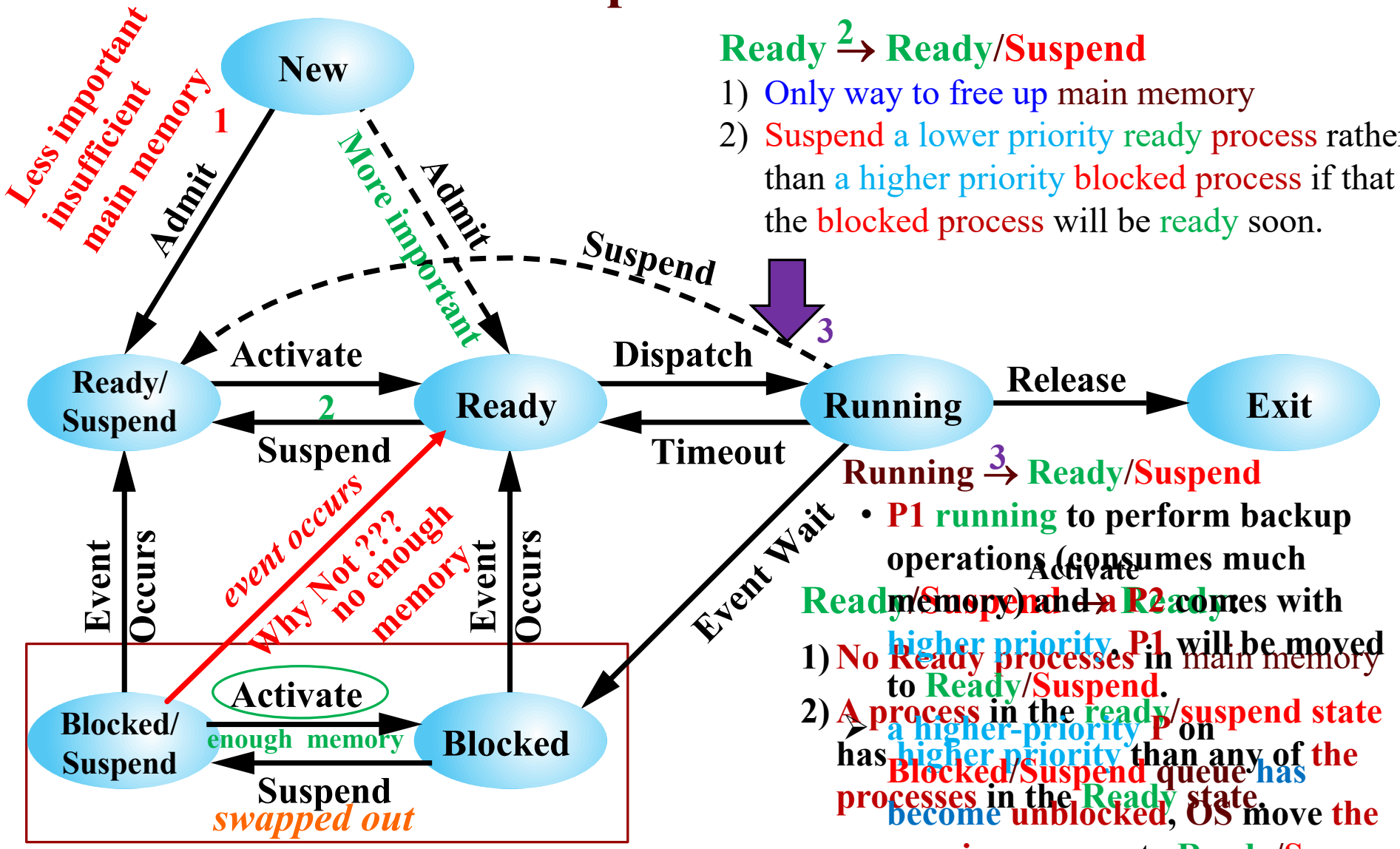
If no blocked processes, the objective is not achieved

Solution: more than one Suspend state

Suspended Processes:The need for swapping

- **Previous Two states**
  - ***Ready***: process *is in main memory and available for execution*.
  - *Blocked*: process *is in main memory and awaiting an event*.
- **Two new states**
  - *Blocked/suspend* (process is in ***secondary storage*** and *awaiting an event*.)
  - ***Ready***/*suspend* (process is in ***secondary storage*** **but** *the event for which it has been waiting* **occurs.**
    - So, it is *available for execution* **when** loaded into *main memory*)

Two Suspend States



(b) With Two Suspend States

Ready $\xrightarrow{2}$ Ready/Suspend

1) Only way to free up main memory
2) Suspend a lower priority ready process rather than a higher priority blocked process if that the blocked process will be ready soon.

Running $\xrightarrow{3}$ Ready/Suspend

- P1 running to perform backup operations (consumes much memory) and a P2 comes with higher priority, P1 will be moved to Ready/Suspend.

Ready/Suspend → Ready:

1) No Ready processes in main memory
2) A process in the ready/suspend state has higher priority than any of the processes in the Ready state.

➢ a higher-priority P on Blocked/Suspend queue become unblocked, OS move the running process to Ready/Suspend queue and free main memory.

The state transition model

The dashed lines in the figure indicate possible but not necessary transitions

1) Blocked → Blocked/Suspend:

- **If there are no ready processes, then at least one blocked process is *swapped out* to make room for another process that is not blocked.**
- **This transition can be made even if there are ready processes available, if the OS determines that the currently running process or a ready process that it would like to dispatch requires more main memory to maintain adequate performance.**

The state transition model

2) **Blocked/Suspend → Ready/Suspend:**

A process in Blocked/Suspend state is moved to Ready/Suspend state when the event for which it has been waiting occurs.

3) Ready/Suspend → Ready:

- **When there are no ready processes in main memory, the OS will need to bring one in to continue execution.**
- **In addition, it might be the case that a process in the Ready/Suspend state has higher priority than any of the processes in the Ready state.**
  - **In that case, the OS designer may dictate that it is more important to get at the higher-priority process than to minimize *swapping*.**

4) Ready → Ready/Suspend:

- ✓ **Normally, the OS would prefer to suspend a blocked process rather than a ready one, because the ready process can now be executed, whereas the blocked process is taking up main memory space and cannot be executed.**
- **However, it may be necessary to suspend a ready process if that is the only way to free up a sufficiently large block of main memory.**
- **Also, the OS may choose to suspend a lower–priority ready process rather than a higher–priority blocked process if that the blocked process will be ready soon.**

Other transitions

5) New → Ready/Suspend and New → Ready:

- **When a new process is created, it can either be added to the Ready queue or the Ready/Suspend queue.**
- **There would often be insufficient room in main memory for a new process hence the use of New → Ready/Suspend.**

6) Blocked/Suspend → Blocked:

- If a process is not ready to execute and is not already in main memory, what is the point of bringing it in? But consider the following scenario:

- There is a process in Blocked/Suspend queue with a higher priority than any of the processes in the Ready/Suspend queue and OS has reason to believe that the blocking event for that process will occur soon.
- It would seem reasonable to bring a blocked process into main memory in preference to a ready process.

7) Running → Ready/Suspend:

- Normally, a running process is moved to Ready state when its time allocation expires.
  - If OS is preempting the process because a higher-priority process on Blocked/Suspend queue has just become unblocked, OS could move the running process directly to Ready/Suspend queue and free some main memory.
  - If there exists a process 1 running to perform backup operations (consumes much memory) and a new process 2 comes with a higher-priority, process 1 will be moved to Ready/Suspend.

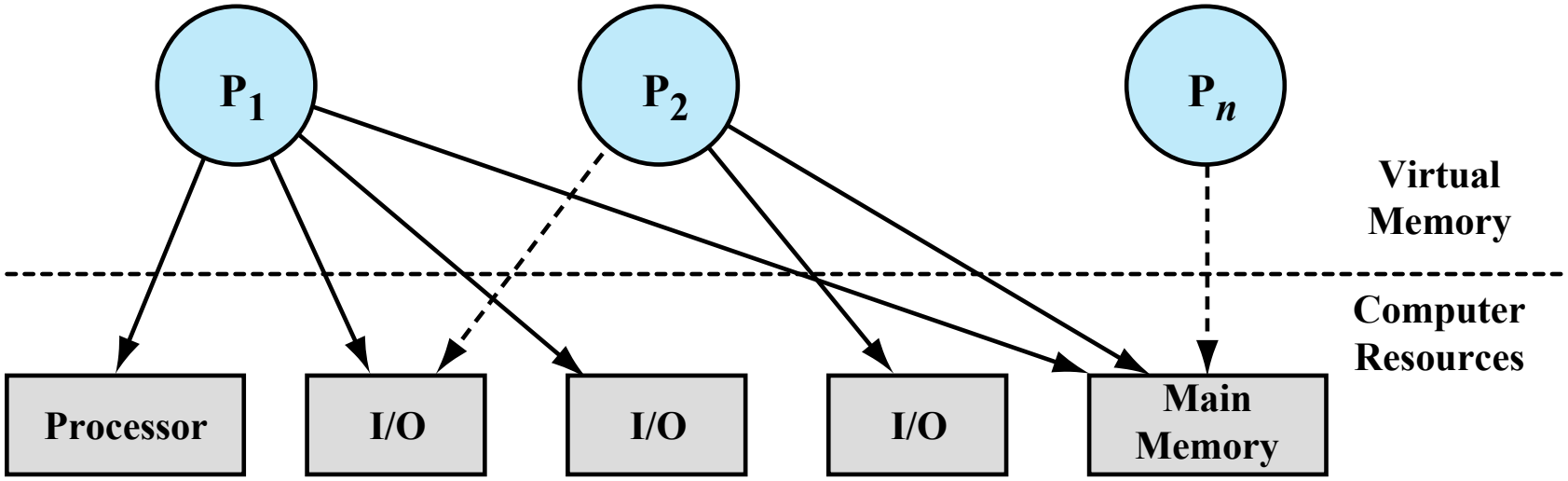
Reasons for process suspension

Swapping	OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	OS may suspend a background or utility process or a process that is suspected of causing a problem. One example of this is deadlock.
Interactive user request	• A user may wish to suspend execution of a program for purposes of debugging • Or there may be a background process that is collecting accounting statistics, which the user may wish to be able to turn on and off.
Timing	A process may be executed periodically (e.g., A program that monitors utilization or user activity) may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend a descendant process. • For example, process A may spawn process B to perform a file read. • Subsequently, process B encounters an error in the file read procedure and reports this to process A. • Process A suspends process B to investigate the cause.

Process Description

- OS **schedules** and **dispatches** processes for execution by the processor, **allocates** resources to processes.
- In a multiprogramming environment, There are a number of processes (P_1, …, P_n) that have been created and exist in virtual memory.

Each process, during execution, needs access to certain system resources(processor, I/O devices, and main memory)



P1: running, part in main memory, has control of **2 I/O devices**
P2: in main memory, blocked waiting for an **I/O device** (allocated to **P1**)
Pn: ***swapped out*** (**suspended**)

Figure 3.9 Processes and Resources (resource allocation at one snapshot in time)

What information does the OS need to **control** processes and **manage** resources for them?

- Operating System control structures
- Process control structures

Operating System Control Structures

- **Information** about the current status of **each process** and **resource**
- **Tables** are constructed for each entity the **OS** manages
- **Four** different types of tables maintained by the **OS**: **memory**, **I/O**, **file**, and **process**

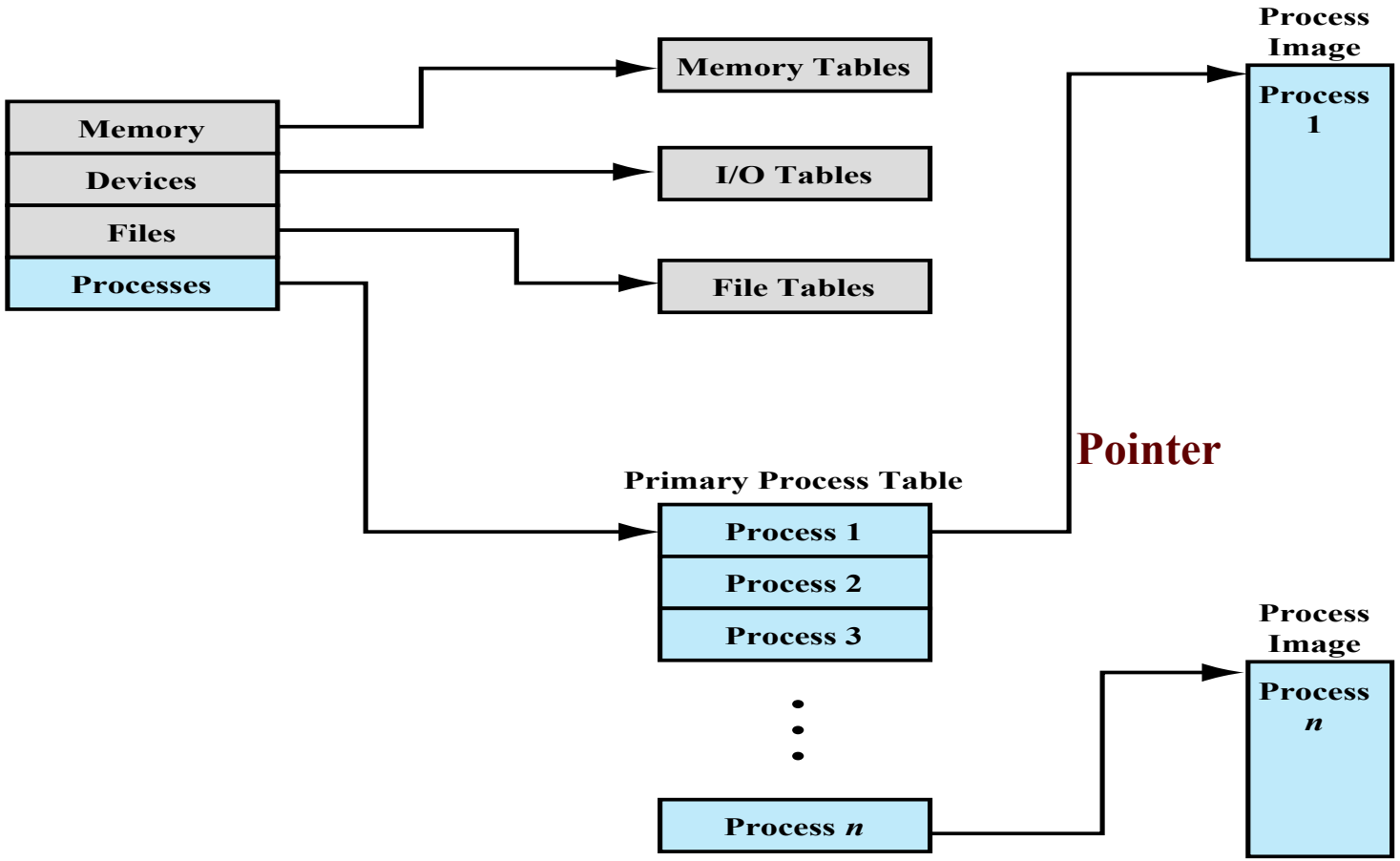


Figure 3.10 General Structure of Operating System Control Tables

Memory tables

- Keep track of both main (real) and secondary (virtual) memory.
- The memory tables must include the following **information**:

1) The allocation of main memory to processes

2) The allocation of secondary memory to processes

3) Protection attributes for access to shared memory regions

4) Information needed to manage virtual memory

I/O tables

- Used by the OS to **manage I/O devices**
- **I/O device** is **available** or **assigned** to a particular process
- If an **I/O operation** is in progress, the OS needs to know

1) Status of **I/O operation**

2) Location in main memory being used as the **source** or **destination** of the **I/O transfer**

File tables

1) Existence of files

2) Location on secondary memory

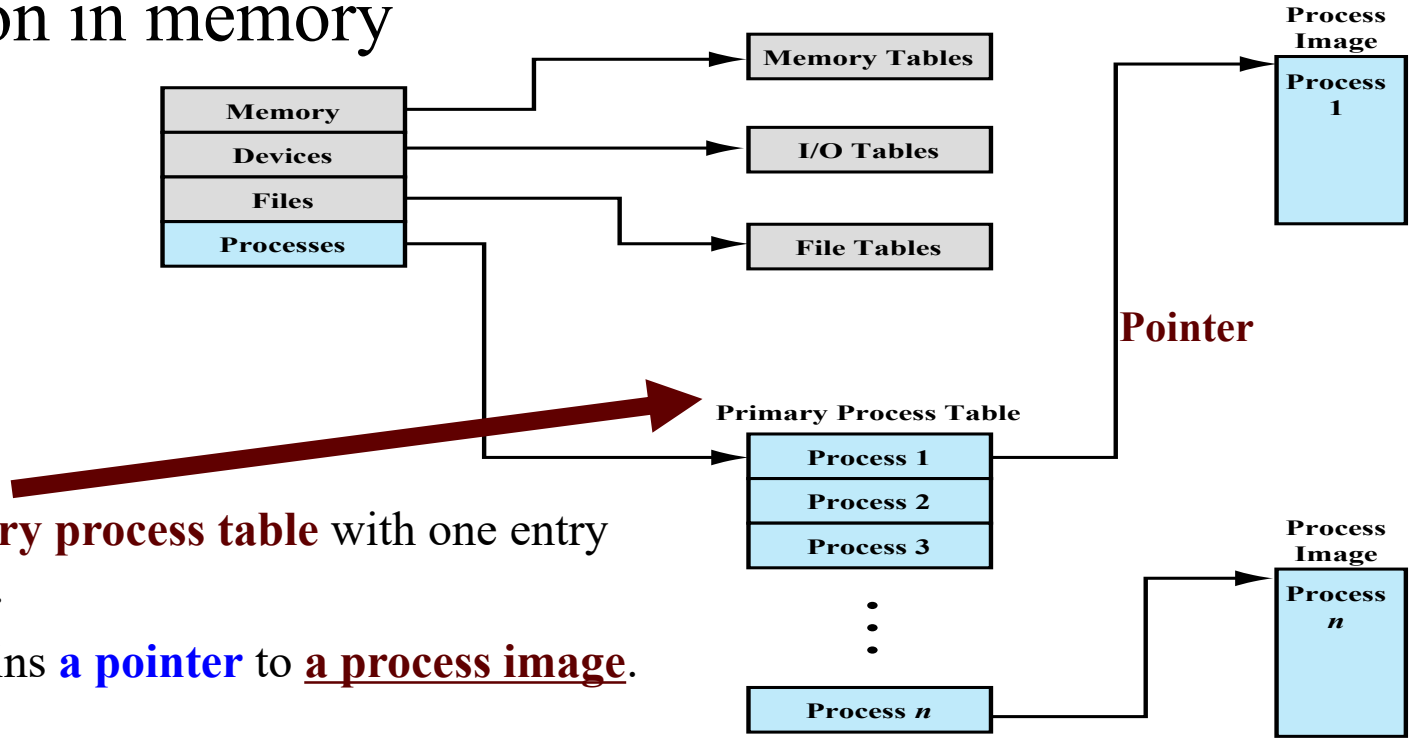
3) Current Status

4) Attributes

This information is maintained by a file-management system

Process tables

1) Where process is located
2) Attributes necessary for its management
   - Process ID
   - Process state
   - Location in memory



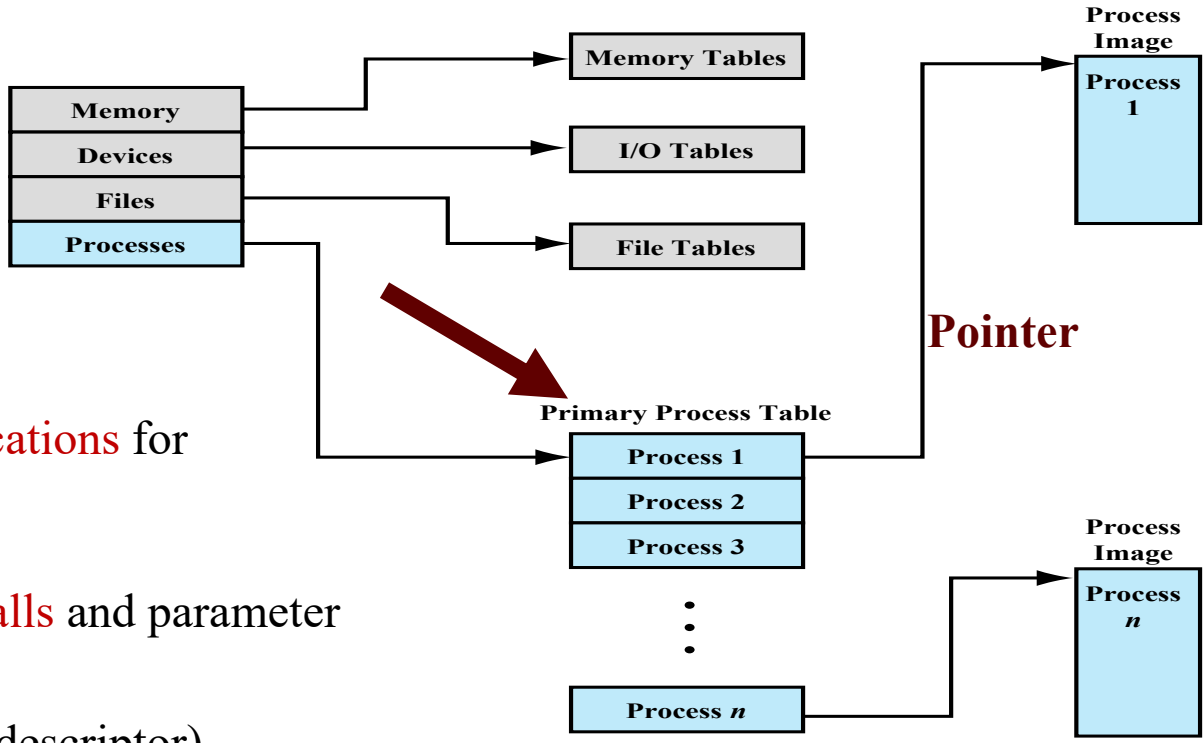
- There is **a primary process table** with one entry for **each process**.
- Each entry contains **a pointer** to **a process image**.

Process Control Structures

For OS to manage and control a process, it must know:

1) Process Location
2) Process Control Block (PCB)
   - The attributes of the process that are necessary for its management (e.g. Process ID and process state).

Process Location



- Process includes:

1) a program/code to be executed
2) associated with a set of data locations for
   – Local and global variables
   – Any defined constants
3) Stack (keep track of function calls and parameter passing between functions)
4) Process control block (process descriptor)
   – Collection of attributes used by OS for process control

Process image

Collection of code, data, stack, and attributes in PCB is referred as **Process image**



- **The process image** is maintained as a contiguous, or continuous, block of memory.
- Figure depicts **the structure of the location information** in the following way:

➢ There is **a primary process table** with one entry for **each process**.

➢ Each entry contains **a pointer** to **a process image**.

Process Control Block (PCB)

The process control block information grouped into three categories:

1) Process identification
2) Processor state information
3) Process control information

1) Process identification

- Identifiers
  - Numeric identifiers that may be stored with the process control block include
    - Identifier of this process (might be index into primary process table)
    - Identifier of the process that created this process (parent process)
    - User identifier responsible for the job

2) Processor State Information (Context Switching)

– Processor registers *while a process is running*

- User-visible registers
- Control and status registers
- Stack pointers

User-Visible Registers (data and address (Index, Segment pointer and Stack pointer))

- referenced by machine language that the processor executes while in **user mode**.
- Typically, there are from 8 to 32 of these registers, although some RISC implementations have over 100.

Control and Status Registers

These are a variety of processor registers that are employed to control the operation of the processor. These include

- *Program counter:* Contains the address of the next instruction to be fetched
- *Condition codes:* Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow)
- *Status information:* Includes interrupt enabled/disabled flags and execution mode

Stack Pointers

- Each process has one or more last-in-first-out (LIFO) system stacks associated with it.
- A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.

Program status word (PSW)

- A register or a set of registers
- Contains condition codes and other status information (interrupt enabled/disabled flags and execution mode)
- Example: the EFLAGS register on Pentium machines

Pentium II EFLAGS Register



ID = Identification flag
VIP = Virtual interrupt pending
VIF = Virtual interrupt flag
AC = Alignment check
VM = Virtual 8086 mode
RF = Resume flag
NT = Nested task flag
IOPL = I/O privilege level
OF = Overflow flag

DF = Direction flag
IF = Interrupt enable flag
TF = Trap flag
SF = Sign flag
ZF = Zero flag
AF = Auxiliary carry flag
PF = Parity flag
CF = Carry flag

Figure 3.11 Pentium II EFLAGS Register

3) Process Control Information

– Scheduling and State Information

This is information that is needed by OS to perform its **scheduling function**.

Typical items of information:

- *Process state:* defines the readiness of the process to be scheduled for execution (e.g., running, ready, waiting, halted).
- *Priority:* One or more fields may be used to describe the scheduling priority of the process. In some systems, several values are required (e.g., default, current, highest-allowable)
- *Scheduling-related information:* This will depend on the scheduling algorithm used. Examples are the amount of time that the process has been waiting and the amount of time that the process executed the last time it was **running**.
- *Event:* Identity of event the process is awaiting before it can be **resumed**

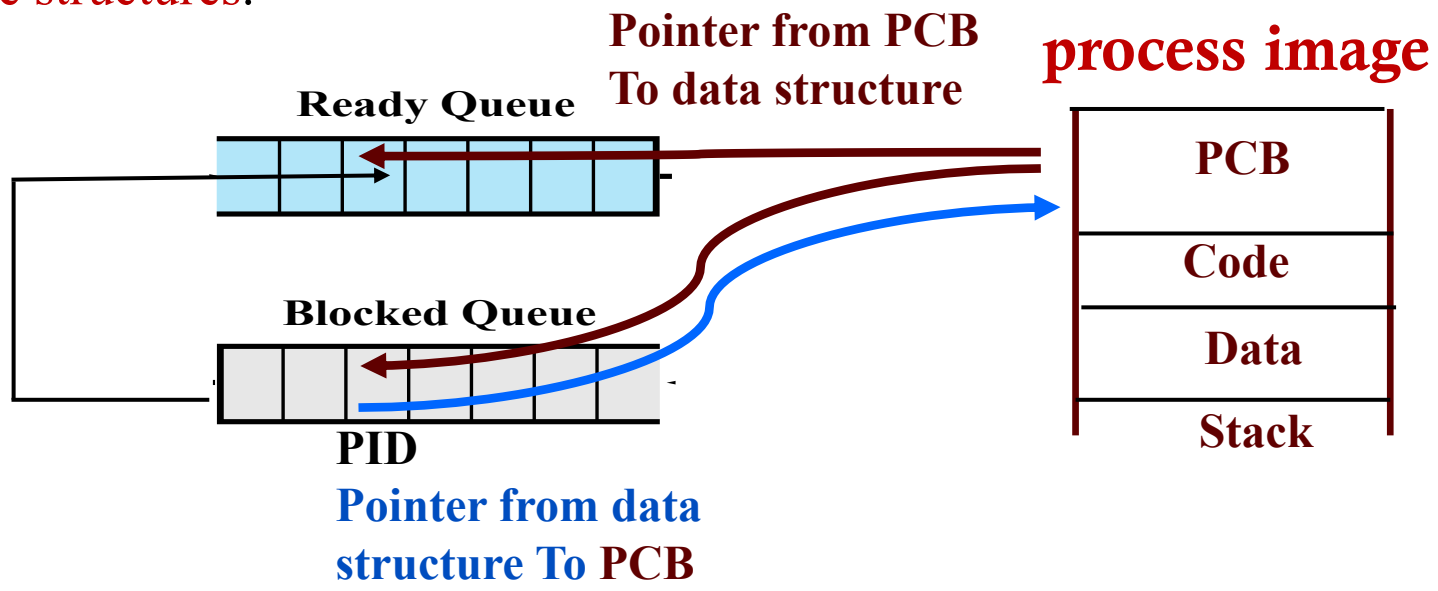
– Data Structuring

- A process may be **linked** to other process in **a queue**, or some other structure.

For example, all processes in a waiting state for a particular priority level may be **linked** in **a queue**.

A process may **exhibit** a parent-child (creator-created) relationship with another process.

➢ The process control block may contain **Pointers** to other processes to **support** these structures.



– Interprocess Communication

- Various flags, signals, and messages may be associated with communication between two independent processes.
  Some or all of this information may be maintained in the process control block.

– Process Privileges

- Processes are granted privileges in terms of the memory that may be accessed and the types of instructions that may be executed.
  In addition, privileges may apply to the use of system utilities and services.

– Memory Management

- Include pointers to segment and/or page tables that describe the virtual memory assigned to this process.

– Resource Ownership and Utilization

- Resources controlled by the process may be indicated, such as opened files.
- A history of utilization of the processor or other resources may also be included; this **information** may be needed by the **scheduler**.

The structure of process images in virtual memory

Each process image consists of a process control block, a user stack, the private address space of the process, and any other address space that the process shares with other processes.

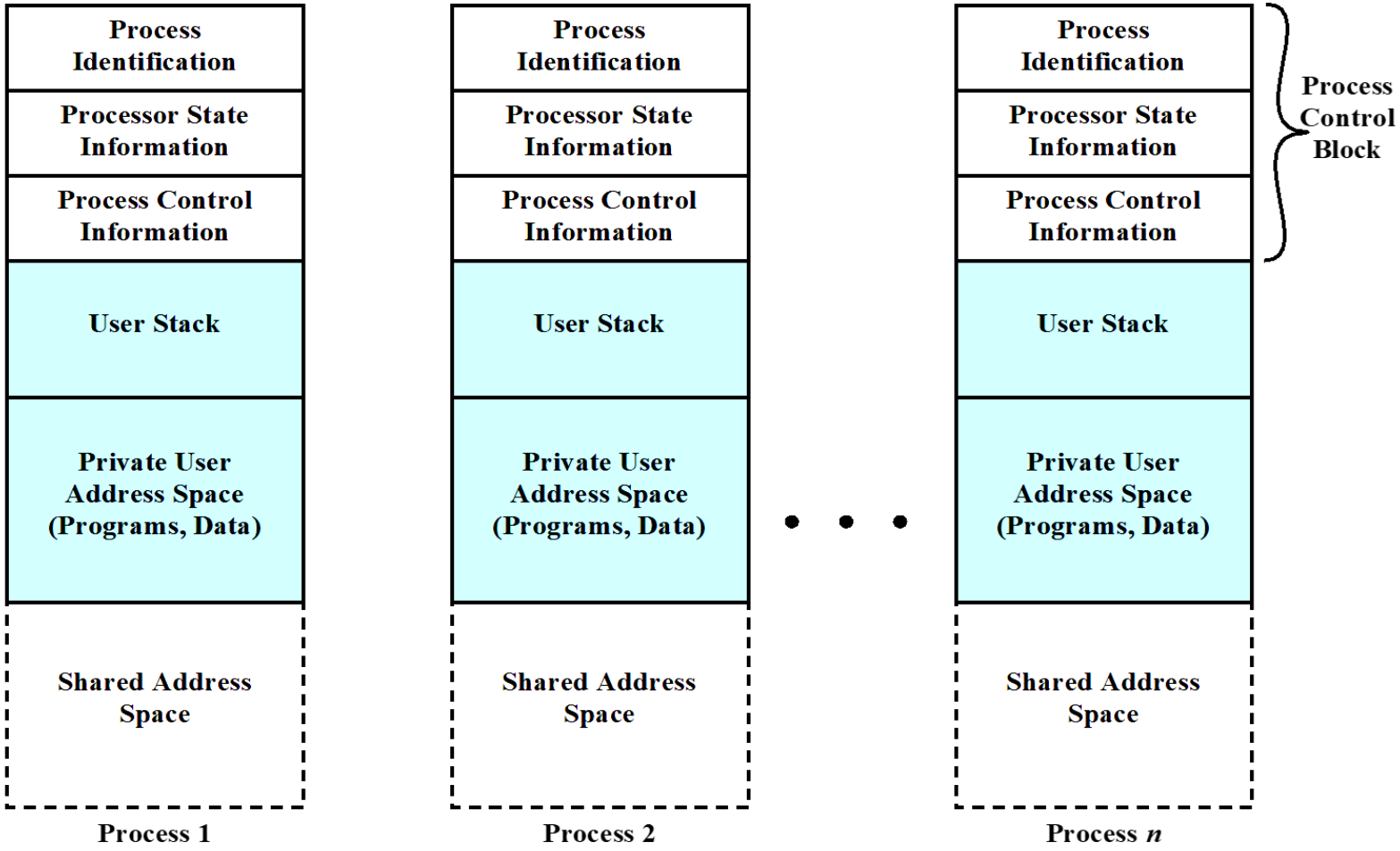


Figure 3.12 User Processes in Virtual Memory

Process Control
The requirements for process control by OS

- Modes of Execution
- Process Creation
- Process Switching
- Execution of the Operating System

Modes of Execution

- Processors support **two modes** of execution Why?
- User mode
  - Less-privileged mode
  - User programs typically execute in this mode
- Kernel mode, System mode, Supervisory or control mode
  - More-privileged mode
  - Kernel of the operating system
  - Examples
    - I/O instructions
    - instructions that relate to memory management.
    - reading or altering a control register such as the PSW
    - interrupt management

Modes of Execution

- A bit in PSW indicates the execution mode
- A user makes a call to an OS service (make use of the hardware)→ the mode is set to kernel mode (typically, by executing an instruction that changes the mode)

➢ **After finishing a system call, The CPU mode is switched back to user mode (by setting the mode bit to 1) before passing control to the user process.**

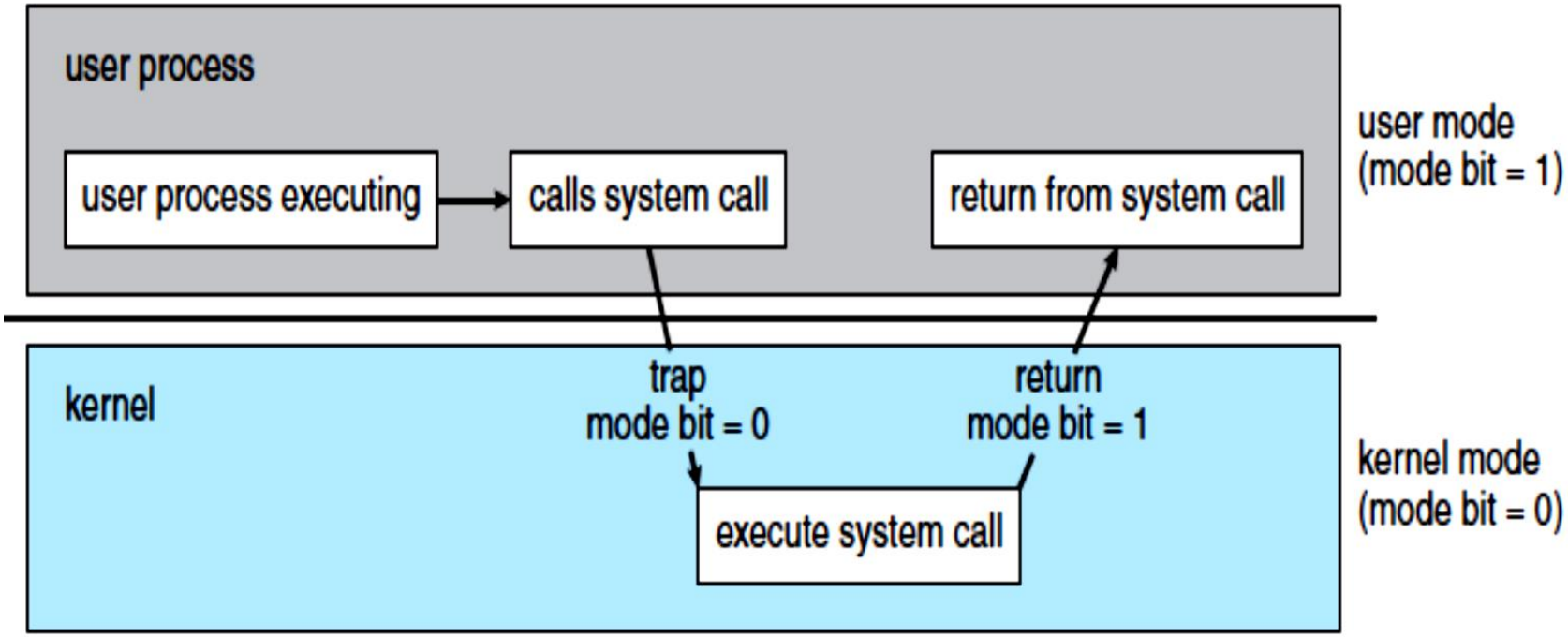


Figure 3.13 Modes of Execution

- Processors support **two modes** of execution Why?

Allows OS to protect itself and key operating system tables, **such as** process control blocks, **from** interference by user programs.

Process Creation

- Assign a unique process identifier
- Allocate space for the process
- Initialize process control block
- Set up appropriate linkages
  - Ex: add new process to linked list used for scheduling queue
- Create or expand other data structures
  - Ex: maintain an accounting file on each process to be used subsequently for billing and/or performance assessment purposes.

Process Switching: When to Switch a Process

- A process switch may occur any time the OS gains control from the currently running process.

1) Interrupts
   - Clock interrupt
     - Process has executed for the maximum allowable time slice
   - I/O interrupt
     - If I/O action has occurred then the OS moves all of the corresponding blocked processes to Ready state (and Blocked/Suspend processes to Ready/Suspend state).
     - The OS must then decide whether to resume execution of the process currently in the Running state, **or** to preempt that process for a higher-priority Ready process.

Process Switching: When to Switch a Process

- Memory fault
  - Processor encounters a virtual memory address reference for a word that is **not in main memory**.
  - The OS must bring in the block (page or segment) of memory containing the reference **from** secondary memory to main memory.
  - The process with the memory fault is placed in **a blocked state**; OS then performs a process switch to resume execution of another process.
  - **After** the desired block is brought into memory, that process is placed in **Ready state**.

Process Switching: When to Switch a Process

2) Trap
   - Error occurred (illegal file access attempt)
   - May cause process to be moved to Exit state
3) Supervisor call
   - Requests an I/O operation, **such as** a file open
   - Generally, the use of a system call results in placing the user process in Blocked state.

Mode Switching

➢ **No change from process to another process**

- If an interrupt is pending, the processor does the following
  - Saves the context of the process that has been interrupted into that process control block of the interrupted program.
  - Sets PC to the starting address of an interrupt-handler program
  - ***Switches from user mode to kernel mode***

    Since the interrupt processing code may include privileged instructions.

Mode Switching

- In most OS, the occurrence of an interrupt does not necessarily mean a process switch.
- It is possible that, **after** the interrupt handler has executed, the currently running process will resume execution. In that case, all that is necessary is to:
  - Save processor state information **when** interrupt occurs
  - Restore that information **when** control is returned to the program that was running.

Steps in a full process switch

- **Change of Process State:** if the currently running process is to be moved to another state (Ready, Blocked, etc.)

1) Save context of processor including PC and other registers
2) Update the process control block of the currently running process
   - This includes changing the state of the process to one of the other states (Ready; Blocked; Ready/Suspend; or Exit).
3) Move process control block of this process to appropriate queue
   - ready, blocked, Ready/Suspend
4) Select another process for execution
5) Update the process control block of the process selected
   - Changing the state of this process to Running

Steps in a full process switch

5) Update memory-management data structures
   - This may be required, depending on how address translation is managed

6) Restore the context of the processor to that which existed at the time the selected process was last switched out of the running state
   - By loading the previous values of PC and other registers

- Thus, the process switch, which involves a state change, requires more effort than a mode switch.
- A mode switch may occur **without changing** the state of the process that is currently in the Running state.

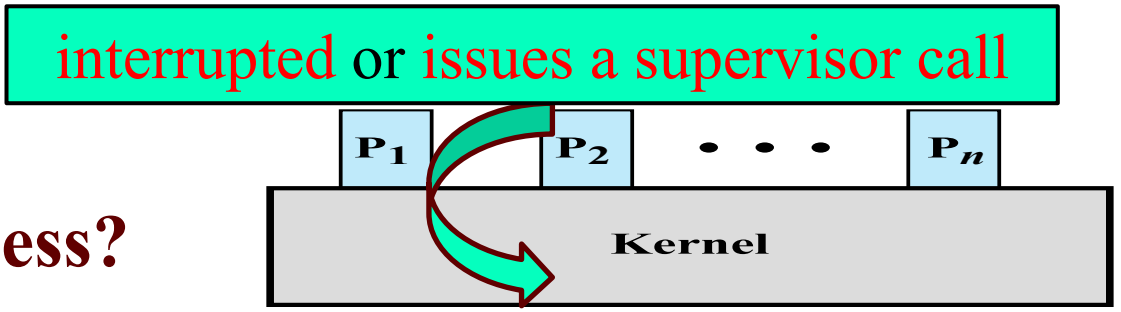
Execution of the Operating System

1) Non-process Kernel (older OS)

- Execute kernel outside of any process
- OS code is executed as a separate entity that operates in privileged mode

- When the currently running process is interrupted or issues a supervisor call, the control is passed to the kernel.

Is the OS a process?



(a) Separate kernel

Figure 3.14 Relationship between Operating System and User Processes

➢ The concept of process is considered to apply only to user programs

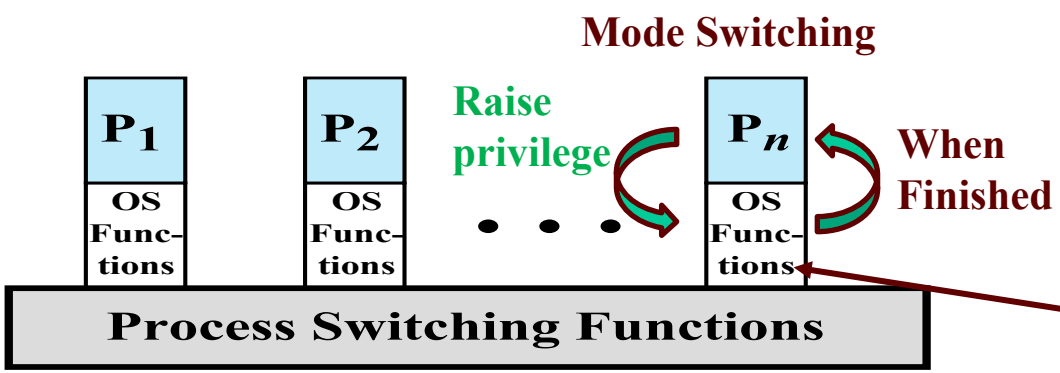
2) Execution Within User Processes(small scale machines:Processing power and Ram)

- OS functions executed within context of a user process

Process executes in privileged mode when executing OS code (Mode Switching i.e. user mode to kernel mode) → Raise privilege

- No switch from process to another process
- E.g. scanf, printf, fileopen (system calls) called within user application → HW needs OS functions.

- After finishing a system call, The CPU mode is switched back to user mode
- Security is low
- OS runs as extension to Processes because no enough Processing power and Storage



OS is a collection of routines that the user calls to perform various functions, executed within the environment of the user's process.

(b) OS functions execute within user processes

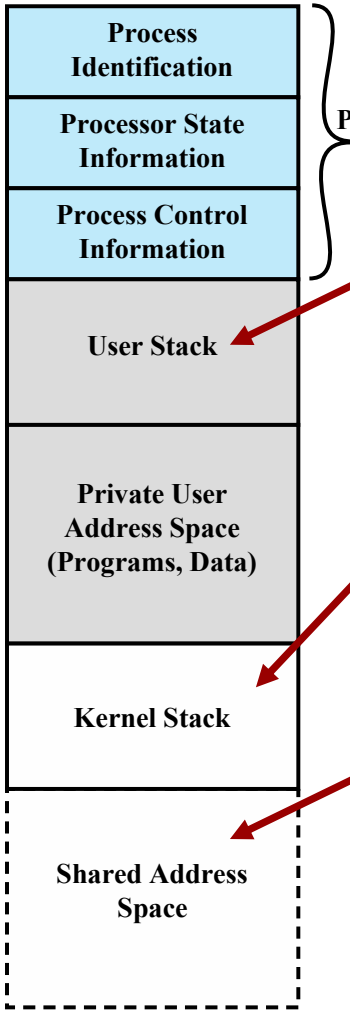
Figure 3.14 Relationship between Operating System and User Processes

Process Switching functions

1) Quantum expired Running → Ready (another process will run)

2) Event wait (I/O) Running → Blocked (another process will run)

Process Image:Mode Switching



User mode to run user functions

Kernel mode to run kernel functions
A separate kernel stack is used to manage calls/returns while the process is in kernel mode.

printf (system calls) called within user application executes inside OS

Contains OS code and data shared by all user processes

Process Image

Mode Switching

OS Executes within User Space

- When an interrupt, trap, or supervisor call occurs, the processor is placed in **kernel mode** and **control** is passed to **OS**.
- **To pass control from a user program to the OS, a mode switch takes place to OS routine.**
- **Execution** continues within the current user process. **Thus, a process switch** is not performed, just **a mode switch within the same process**.
- If the **OS**, upon **completion** of its work, **determines** that **the current process** should **continue to run**, then **a mode switch resumes** the interrupted program within **the current process**.

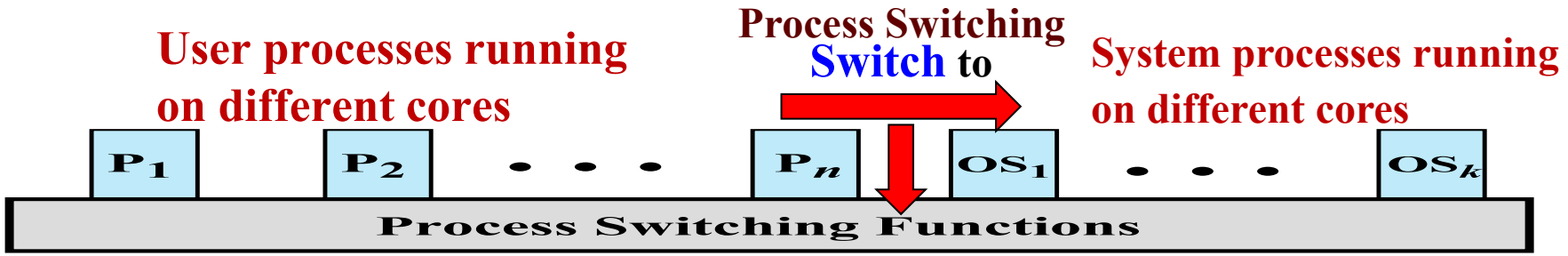
Execution Within User Processes

This is one of the key advantages of this approach:

- **A user program has been interrupted to employ some OS routine, and then resumed, and all of this has occurred without incurring the penalty of two process switches.**

3)Process-Based OS (large scale machines: Processing power (Cores) and Ram)

– Major **kernel functions** are separate processes



(c) OS functions execute as separate processes

Figure 3.14 Relationship between Operating System and User Processes

- **Advantages**

1) **Security is good (no need to change user process to privileged mode)**
   - **Only switch from user process to system process**
   - **E.g. user process (office) open file → switch to system process to open file (no Mode Switching)**
2) Useful in a multiprocessor or multicomputer environment (some of the OS services can be shipped out to dedicated processors, improving performance)

Execution of the Operating System

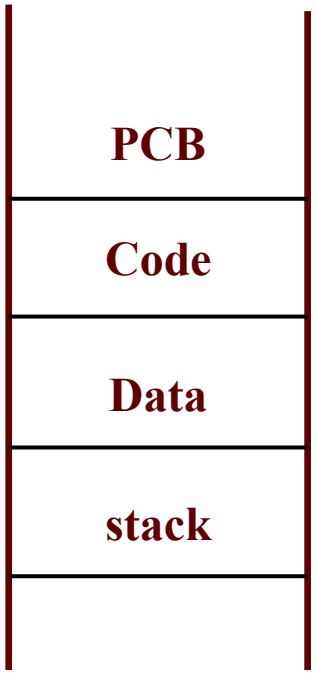
3)Process-Based OS (large scale machines: Processing power (Cores) and Ram)

- Major **kernel functions** are separate processes

Some noncritical operating system functions are implemented as **separate processes**.

- For example, **a monitor program** that records the level of utilization of various resources (processor, memory, channels) and the rate of progress of the user.
- Because this program does not provide a particular service to any active process, it can only be invoked by **OS**.
- As a process, the function can run at an assigned priority level and be interleaved with other processes under dispatcher control.

Process Image: Process Switching



Only one stack why?
Process is user process or system process
If user process → user functions
If system process → system functions run as OS